# Table des matières

**1 Introduction** **2**

**2 Choix d'implémentation et Structures de données** **2**
2.1 Représentation de la Base (Graphe) . . . . . 2
2.2 File de Priorité . . . . . 2
2.3 Table de Réservation (Phase 2) . . . . . 2

**3 Description des Algorithmes** **2**
3.1 Phase 1 : Recherche du plus court chemin (Dijkstra) . . . . . 2
3.2 Phase 2 : Ordonnancement . . . . . 3

**4 Tests et Résultats** **4**
4.1 Validation de la Phase 1 (Dijkstra) . . . . . 4
4.2 Validation de la Phase 2 (Ordonnancement) . . . . . 4

**5 Problèmes Techniques et Solutions** **5**

**6 Limites et Conclusion** **5**

# 1 Introduction

Le but de ce projet est de concevoir un système de planification de déplacements pour une base martienne. Cette base est constituée de modules reliés par des tunnels ne permettant le passage que d'une personne à la fois.

Le projet se décompose en plusieurs phases de complexité croissante :

- **Phase 1 :** Gestion du déplacement d'un individu unique en minimisant le temps de parcours.
- **Phase 2 :** Gestion de plusieurs individus avec des itinéraires pré-définis, en évitant les collisions dans les tunnels et en minimisant le temps total
- **Phase 3 :** Planification autonome des itinéraires pour plusieurs individus (non traité dans ce rapport, l'accent étant mis sur les deux premières phases implémentées).

Ce rapport présente les choix de structures de données, détaille les algorithmes utilisés (Dijkstra et Ordonnancement Glouton) et analyse les résultats obtenus lors des tests.

# 2 Choix d'implémentation et Structures de données

Le choix du langage OCaml nous a orientés vers des structures de données immuables et persistantes, favorisant la récursion, dont l'implantation se repose au maximum sur la bibliothèque standard d'Ocaml, via l'utilisations des modules Map et Set.

## 2.1 Représentation de la Base (Graphe)

La base est représentée comme un graphe non orienté pondéré. Nous avons utilisé le module `Map` de la librairie standard.

- **Tunnels :** Une `Map` associant chaque module (chaîne de caractères) à un `Set` de ses voisins accessibles.
- **Poids :** Une `Map` dont la clé est une paire de modules $(M_1, M_2)$ dont la fonction de comparaison repose sur l'ordre lexicographique, et la valeur est le temps de traversée (entier).

## 2.2 File de Priorité

Pour l'algorithme de Dijkstra (Phase 1), une file de priorité efficace est indispensable. Nous avons implémenté un tas binaire (Heap) via le foncteur `MakeHeap` plutôt qu'une simple liste triée. Cela permet d'effectuer les opérations d'ajout et d'extraction du minimum avec une meilleure complexité temporelle.

## 2.3 Table de Réservation (Phase 2)

Pour gérer les conflits d'accès aux tunnels, nous avons introduit une structure `ReservationsMap`. Elle associe à chaque tunnel une liste d'intervalles de temps occupés $[t_{dbut}, t_{fin}]$. Cette structure permet de vérifier rapidement la disponibilité d'un tunnel pour un créneau donné.

# 3 Description des Algorithmes

## 3.1 Phase 1 : Recherche du plus court chemin (Dijkstra)

Pour trouver le chemin le plus rapide entre deux modules, nous avons implémenté l'algorithme de Dijkstra. L'approche est fonctionnelle récursive, utilisant des accumulateurs pour les distances et les parents.

**Algorithm 1:** Algorithme de Dijkstra (Phase 1)

```
Input: Graphe, Départ, Arrivée
Output: Distance, Chemin
Initialisation :
- Distances : Map (Départ → 0, autres → ∞);
- Parents : Map vide;
- File : File de priorité avec (Départ, 0);
Fonction dijkstra (Distances, Parents, File) is
    if File est vide then
        Lever Exception Not_found (aucun chemin);
    else
        (u, DistU), ResteFile ← extraire minimum de File;
        if u = Arrivée then
            return (DistU, reconstruire_chemin(Parents, Arrivée));
        end
        if DistU > Distances[u] then
            dijkstra(Distances, Parents, ResteFile);
        else
            foreach Voisin de u do
                NouvDist ← DistU + Poids(u, Voisin);
                if NouvDist < Distances[Voisin] then
                    Mettre à jour Distances[Voisin] ← NouvDist;
                    Mettre à jour Parents[Voisin] ← u;
                    Ajouter (Voisin, NouvDist) à ResteFile;
                end
            end
            dijkstra(Distances, Parents, ResteFile);
        end
    end
end
```

### 3.2 Phase 2 : Ordonnancement

Pour la gestion de plusieurs explorateurs, nous utilisons une stratégie gloutonne. L'idée est de prioriser les explorateurs ayant le trajet le plus long afin qu'ils ne soient pas bloqués de manière répétée par des trajets courts, optimisant ainsi le temps global.

**Algorithm 2:** Algorithme d'Ordonnancement (Phase 2)

```
Input: Liste des Chemins
Output: Liste (Chemin, Horaires), TempsTotal
Préparation :
- Calculer la longueur totale de chaque chemin (fonction path_length);
- Trier les explorateurs par priorité décroissante (plus long chemin en premier);
- Initialiser Reservations (Map vide) et Resultats (vide);
Ordonnancement :
foreach Explorateur dans ListeTriée do
    TempsCourant ← 0;
    foreach Tunnel (A, B) du chemin do
        Durée ← Poids(A, B);
        /* Recherche récursive du premier créneau libre                    */
        while Conflit dans Reservations pour (A, B) entre [TempsCourant,
          TempsCourant + Durée] do
            TempsCourant ← Fin de l'intervalle conflictuel;
        end
        Ajouter l'intervalle [TempsCourant, TempsCourant + Durée] dans Reservations
          pour (A, B);
        Enregistrer le temps de départ pour ce segment;
        TempsCourant ← TempsCourant + Durée;
    end
    Ajouter (Chemin, Horaires) aux Resultats;
    Mettre à jour TempsGlobal = Max(TempsGlobal, TempsArrivée);
end
Finalisation :
Trier Resultats par index croissant (ordre initial);
return (Resultats, TempsGlobal);
```

# 4 Tests et Résultats

Nous avons effectué quelques tests pour valider la robustesse de nos algorithmes. Les fichiers de tests sont structurés pour vérifier des cas limites et des propriétés spécifiques.

## 4.1 Validation de la Phase 1 (Dijkstra)

Les tests de la phase 1 (fichiers `base_ex`) ont permis de vérifier les points suivants :

- **Cycles et détours :** L'algorithme ne boucle pas infiniment et trouve le chemin optimal même en présence de cycles (ex : `base_ex8`).
- **Poids nuls :** Gestion correcte des tunnels à durée 0 (ex : `base_ex7`, `base_ex8`).
- **Absence de chemin :** Levée correcte de l'exception `Not_found` si la destination est inatteignable (ex : `base_ex4`).
- **Chemins multiples :** Stabilité du choix lorsque plusieurs chemins ont le même poids (ex : `base_ex9`).

## 4.2 Validation de la Phase 2 (Ordonnancement)

Les tests de la phase 2 ont mis en évidence la gestion des conflits temporels :

- **Priorité :** L'explorateur avec le trajet le plus long passe en priorité lors d'un départ simultané, réduisant l'attente globale (ex : `base_ex2`).

— **Sens unique temporel :** Un tunnel occupé dans le sens $A \to B$ bloque aussi le sens $B \to A$ (ex : `base_ex4`).
— **Parallélisme :** Des groupes sans tunnels communs naviguent en parallèle sans s'attendre, le temps total n'est pas la somme des temps individuels (ex : `base_ex6`).
— **Attente forcée :** Un explorateur non prioritaire peut s'insérer avant un prioritaire si le créneau est libre suffisamment tôt, optimisant l'utilisation des ressources (ex : `base_ex7`).

# 5 Problèmes Techniques et Solutions

**Représentation des tunnels non orientés :** Une difficulté majeure a été la gestion des clés dans la `Map` des poids. Le tunnel $A \to B$ est le même que $B \to A$. Pour éviter les duplications ou les erreurs de recherche, nous avons implémenté une fonction simple de normalisation `get_tunnel_key` qui ordonne lexicographiquement les deux modules avant de créer la clé. Ainsi, $(A, B)$ et $(B, A)$ produisent toujours la clé $(A, B)$.

# 6 Limites et Conclusion

Le programme répond efficacement aux exigences des Phases 1 et 2. L'utilisation d'algorithmes fonctionnels purs garantit la sûreté du code.

Cependant, notre solution présente certaines limites :

— **Approche Gloutonne :** Pour la Phase 2, nous utilisons une heuristique (priorité au plus long chemin). Bien que performante, elle ne garantit pas l'optimum global absolu pour tous les cas de figure.
— **Limites de la récursion :** Sur de grands graphes, la récursion pourrait atteindre la limite de la pile d'appel, et bien que l'usage de la récursion terminale limite ce risque, l'algorithme pourra difficilement supporter des graphes très lourds.
— **Phase 3 :** L'autonomie complète (calcul des itinéraires ET ordonnancement simultané) n'a pas été pleinement intégrée, le système actuel reposant sur des chemins pré-calculés pour l'ordonnancement.

En conclusion, ce projet a permis de mettre en œuvre des structures de données avancées en OCaml et de résoudre un problème d'optimisation concret via une modélisation par graphes efficace. Il a permis de mettre en oeuvres les différentes structures de données (notamment mais sans s'y limiter, les Map, les Set et le module Priority qui à été codé en TP) et techniques (notamment l'usage de fold sur différentes structures) vues en cours dans un cas concret.